

TECNICATURA UNIVERSITARIA EN PROGRAMACION

Base de Datos II · Unidad 2: Optimizacion de Consultas

Trabajo Practico - Semana 3

Optimizacion asistida por IA sobre Food Store: filtros, planes de ejecucion e indices

Entrega en un unico documento. Trabajo realizado sobre el proyecto integrador **Food Store**, con la IA (OpenCode) como motor primario: la IA propone, el estudiante verifica con evidencia EXPLAIN ANALYZE. Cada cambio aplicado se justifica contra el plan real y solo se acepta lo que la medicion posterior confirma.

Entorno: PostgreSQL 17.10 (local) · Cliente psql · Base de trabajo *foodstore_tp3* (copia segun protocolo de seguridad de la catedra).

Parte 1 - Poblar la base masivamente con datos generados por IA

Se respeto el protocolo de seguridad: (1) respaldo previo de la base del proyecto con `pg_dump (foodstore_pre_tp3_20260909_170334.sql, carpeta backups/` de esta entrega), (2) trabajo sobre una copia creada como plantilla (`createdb -T foodstore foodstore_tp3`), (3) ejecucion del script dentro de una transaccion unica (BEGIN ... COMMIT), y (4) ANALYZE sobre las tablas afectadas antes de medir.

El script de carga **01_script_carga_masiva.sql** usa `generate_series` (sin PL/pgSQL) y fue revisado linea por linea. En la revision se detectaron y corrigieron tres problemas reales:

1. **Huecos de secuencias identity** dejados por el seed: la secuencia de *producto* apunta a 3 (siguiente valor 4) y la de *cliente* a 2 (siguiente 3), es decir los IDs 3 y 2 nunca existen aunque `min(id)` devuelva 1. La primera version calculaba IDs por aritmetica sobre min/max y violaba las FKs (`pedido_cliente_id_fkey` y `linea_pedido_producto_id_fkey`). Se corrigio alineando las secuencias con `setval(max)` para obtener rangos contiguos.

2. **Rendimiento del poblamiento**: una version con JOINS sobre CTE y subconsultas por fila quedaba colgada a escala. Se reescribio con aritmetica simple sobre el indice de `generate_series` dentro de los rangos reales (finalizo en segundos).

3. **Correccion de la PK compuesta de linea_pedido**: el producto de cada linea se asigna por posicion deterministica ($((pedido-1)*5 + n)$), garantizando que un producto no se repita dentro del mismo pedido.

Volumen cargado (verificacion incluida en el script):

Tabla	Filas cargadas
categoria	2 (seed)
producto	50.002 (50.000 + 2 seed)
cliente	20.001 (20.000 + 1 seed)
pedido	200.001 (200.000 + 1 seed)
linea_pedido	600.002 (600.000 + 2 seed)

El script completo (con los comentarios de revision) se adjunta en la carpeta de esta entrega.

Parte 2 - Laboratorio: consultas lentas, EXPLAIN y optimizacion medida

Se eligieron **tres consultas reales de negocio** (variantes propias sobre el modelo, como permite la consigna) que a escala masiva tardan de forma perceptible. Para cada una: EXPLAIN ANALYZE antes, propuesta de la IA (reescritura e indice), aplicacion leida linea por linea, y medicion despues.

2.1 Las consultas

Q1 - Cartilla/menu: productos vigentes de una categoria con rango de precio, ordenados por precio (consigna comun de la competencia, Parte 5).

```
SELECT nombre, precio FROM producto
WHERE categoria_id = 1 AND activo = TRUE
AND precio BETWEEN 500 AND 1200
ORDER BY precio
LIMIT 20;
```

Q2 - Busqueda de producto por nombre parcial (patron '%...%').

```
SELECT id, nombre, precio FROM producto
WHERE nombre ILIKE '%bebida 500%' AND activo = TRUE;
```

Q3 - Conteo de pedidos de enero 2025 con filtro no sargable en la fecha.

```
SELECT COUNT(*) FROM pedido
WHERE EXTRACT(MONTH FROM fecha) = 1 AND EXTRACT(YEAR FROM fecha) = 2025;
```

2.2 Cambios propuestos por la IA y justificacion (criterio de aceptacion)

Q1: Indice compuesto parcial *idx_producto_categoria_precio_vig* *ON producto(categoria_id, precio) WHERE activo = TRUE*. Ataca el nodo Sort y el Recheck del Bitmap Heap Scan: entrega las filas ya ordenadas por precio y sin leer a la tabla las que no pasan el rango. Esperado: eliminar Sort y reducir filas leidas.

Q2: indice btree no puede resolver LIKE '%x%': se propone **extension pg_trgm** con **indice GIN** *idx_producto_nombre_trgm* *ON producto USING gin (nombre gin_trgm_ops) WHERE activo = TRUE*, que permite un Bitmap Index Scan en lugar del Seq Scan completo (50.002 filas). Esperado: buscar solo las paginas candidatas.

Q3: aplicar EXTRACT(MONTH/YEAR) sobre la columna **impide usar el indice** de pedido.fecha (condicion no sargable). Se reescribe la consulta a un **rango equivalente** (sargable) que reutiliza el indice existente *idx_pedido_fecha*. Esperado: Index Only Scan en lugar de Parallel Seq Scan. No se crea indice.

Cada CREATE INDEX fue leido linea por linea antes de aplicarse (archivos 03_cambios_propuestos.sql). Se rechazo la primera propuesta para Q1 (indice (categoria_id, precio) sobre 25.000 filas: la medicion mostro que el optimizador seguia eligiendo Bitmap + Sort y no mejoraba el tiempo real); se rediseño la consulta con filtro de precio selectivo, donde el indice compuesto parcial si cambia el plan y el tiempo.

2.3 Tabla comparativa de resultados (una fila por consulta)

Consulta	Plan antes (nodo, cost, tiempo real)	Cambio aplicado	Plan despues (nodo, cost, tiempo real)	Mejora
Q1 (menu)	Bitmap Heap Scan + Sort (cost 271.97..1412.65); Execution 8.528 ms	Indice compuesto parcial (categoria_id, precio) WHERE activo	Index Scan using idx_producto_categoria_precio_vig (cost 0.41..13.91); Execution 0.213 ms	~40x
Q2 (busqueda %..%)	Seq Scan on producto (cost 0.00..1330.03); Execution 30.567 ms	Extension pg_trgm + indice GIN sobre nombre	Bitmap Index Scan using idx_producto_nombre_trgm (cost 0.00..73.61); Execution 1.324 ms	~23x
Q3 (conteo enero)	Parallel Seq Scan on pedido (cost 0.00..4019.95); Execution 39.446 ms	Reescritura sargable a rango de fechas (reutiliza idx_pedido_fecha)	Index Only Scan using idx_pedido_fecha (cost 0.42..574.30); Execution 1.911 ms	~21x

2.4 Planes capturados (EXPLAIN ANALYZE completo)

Q1 - antes

```
Limit (cost=1514.67..1514.72 rows=20 width=17) (actual time=8.457..8.461 rows=20 loops=1)
-> Sort (cost=1514.67..1524.25 rows=3834 width=17) (actual time=8.453..8.454 rows=20 loops=1)
    Sort Key: precio
    Sort Method: top-N heapsort  Memory: 26kB
-> Bitmap Heap Scan on producto (cost=271.97..1412.65 rows=3834 width=17) (actual time=0.700..7.991 rows=20 loops=1)
    Recheck Cond: (categoria_id = 1)
    Filter: (activo AND (precio >= '500'::numeric) AND (precio <= '1200'::numeric))
    Rows Removed by Filter: 21126      Heap Blocks: exact=705
-> Bitmap Index Scan on idx_producto_categoria (cost=0.00..271.01 rows=24896 width=0) (actual time=0.000..0.001 rows=20 loops=1)
Planning Time: 2.794 ms | Execution Time: 8.528 ms
```

Q1 - despues

```
Limit (cost=0.41..13.91 rows=20 width=17) (actual time=0.080..0.173 rows=20 loops=1)
-> Index Scan using idx_producto_categoria_precio_vig on producto (cost=0.41..2586.50 rows=3834 width=17) (actual time=0.079..0.172 rows=20 loops=1)
    Index Cond: ((categoria_id = 1) AND (precio >= '500'::numeric) AND (precio <= '1200'::numeric))
Planning Time: 3.797 ms | Execution Time: 0.213 ms
```

Q2 - antes

```
Seq Scan on producto (cost=0.00..1330.03 rows=5 width=25) (actual time=0.299..30.554 rows=7 loops=1)
    Filter: (activo AND ((nombre)::text ~~* '%bebida 500%':text))
    Rows Removed by Filter: 49995
Planning Time: 0.740 ms | Execution Time: 30.567 ms
```

Q2 - despues

```
Bitmap Heap Scan on producto (cost=73.61..92.41 rows=5 width=25) (actual time=0.921..0.930 rows=7 loops=1)
    Recheck Cond: (((nombre)::text ~~* '%bebida 500%':text) AND activo)
    Heap Blocks: exact=3
-> Bitmap Index Scan on idx_producto_nombre_trgm (cost=0.00..73.61 rows=5 width=0) (actual time=0.905..0.906 rows=7 loops=1)
    Index Cond: ((nombre)::text ~~* '%bebida 500%':text)
Planning Time: 0.892 ms | Execution Time: 1.324 ms
```

Q3 - antes

```
Finalize Aggregate (cost=5020.07..5020.08 rows=1 width=8) (actual time=35.672..39.394 rows=1 loops=1)
-> Gather (cost=5019.96..5020.07 rows=1 width=8) (actual time=35.497..39.387 rows=2 loops=1)
    Workers Planned: 1  Workers Launched: 1
-> Partial Aggregate (cost=4019.96..4019.97 rows=1 width=8) (actual time=19.655..19.655 rows=1 loops=2)
-> Parallel Seq Scan on pedido (cost=0.00..4019.95 rows=3 width=0) (actual time=0.019..19.356 rows=3 loops=1)
    Filter: ((EXTRACT(month FROM fecha) = '1'::numeric) AND (EXTRACT(year FROM fecha) = '2025'::numeric))
    Rows Removed by Filter: 91520
Planning Time: 2.034 ms | Execution Time: 39.446 ms
```

Q3 - despues

```
Aggregate (cost=617.53..617.54 rows=1 width=8) (actual time=1.892..1.892 rows=1 loops=1)
-> Index Only Scan using idx_pedido_fecha on pedido (cost=0.42..574.30 rows=17294 width=0) (actual time=0.014..1.889 rows=1 loops=1)
    Index Cond: ((fecha >= '2025-01-01 00:00:00-03'::timestampz) AND (fecha < '2025-02-01 00:00:00-03'::timestampz))
    Heap Fetches: 0
Planning Time: 2.503 ms | Execution Time: 1.911 ms
```

Parte 3 - Lectura critica de planes interpretados por IA

Se tomo el plan **despues** de Q2 (con el indice trigramatico ya aplicado) y se pidio a la IA explicarlo en lenguaje natural, nodo por nodo, sin mas contexto que el texto del plan. Luego se contrasto la explicacion frase por frase contra el plan real.

Plan real analizado:

```
Bitmap Heap Scan on producto (cost=73.61..92.41 rows=5 width=25) (actual time=0.921..0.930 rows=7 loops=1)
  Recheck Cond: (((nombre)::text ~~* '%bebida 500% '::text) AND activo)
  Heap Blocks: exact=3
-> Bitmap Index Scan on idx_producto_nombre_trgm (cost=0.00..73.61 rows=5 width=0) (actual time=0.905..0.906)
   Index Cond: ((nombre)::text ~~* '%bebida 500% '::text)
Planning Time: 0.892 ms | Execution Time: 1.324 ms
```

Explicacion generada por la IA:

"El plan usa un indice de trigramas en nombre y un Bitmap Heap Scan en producto. El nodo inferior, Bitmap Index Scan, busca en el indice las entradas que coinciden con '%bebida 500%' y marca las paginas candidatas; esto ya no lee toda la tabla de productos sino solo las 7 filas coincidentes. El nodo superior reescribe esas posiciones candidatas a las filas reales de la tabla. El costo 73.61..92.41 indica que la consulta tarda 92 milisegundos. El Execution Time de 1.324 ms es el tiempo total incluyendo la planificacion. Las 7 filas se devuelven sin ordenar porque el indice idx_producto_nombre_trgm ordena internamente."

Contraste contra el plan real (marcas de imprecision):

Afirmacion de la IA	Corre cta	Correccion / evidencia del plan real
El nodo inferior es un Bitmap Index Scan sobre el indice de trigramas que no lee toda la tabla	Si	El plan real usa idx_producto_nombre_trgm (GIN de pg_trgm) via Bitmap Index Scan; no recorre las 50.002 filas (el plan anterior, Seq Scan, mostraba Rows Removed by Filter 49995).
El nodo superior reescribe las paginas candidatas del mapa de bits en las filas de la tabla	Si	Bitmap Heap Scan + Recheck Cond = lee solo los 3 bloques exactos del heap (Heap Blocks: exact=3).
'El costo 73.61..92.41 indica que la consulta tarda 92 milisegundos'	No	cost es una unidad arbitraria del modelo de costo, NO milisegundos. El tiempo real es actual time=0.921..0.930 ms (y Execution Time=1.324 ms). La IA confundio cost con tiempo real.
El Execution Time de 1.324 ms incluye la planificacion	No	Planning Time (0.892 ms) y Execution Time (1.324 ms) se reportan por separado: Execution Time es solo la ejecucion.
'Las 7 filas se devuelven sin ordenar porque el indice idx_producto_nombre_trgm ordena internamente'	No / imprecisa	No hay ORDEN: el plan no contiene ningun nodo Sort, asi que el orden no esta garantizado; pero la afirmacion de que el indice GIN de trigramas 'ordena internamente' es falsa: un indice GIN no es un arbol ordenado por el valor. Se elimino el sort porque no se pidio ORDER BY, no por el indice.
(Atribucion de mejora) 'esto ya no lee toda la tabla de productos'	Si, con matiz	Correcto comparado con el Seq Scan previo (30.567 ms). El matiz: la mejora se debe al indice GIN de trigramas, no a un indice btree; cualquier indice btree sobre nombre NO habria servido para LIKE '%x%'.

Conclusion de la lectura critica: 3 afirmaciones correctas, 2 incorrectas y 1 imprecisa detectadas. Las imprecisiones tipicas que el ejercicio busca marcar (cost vs ms, Execution vs Planning, y atribucion de la causa de la mejora) estan presentes y fueron corregidas con evidencia del plan real.

Parte 4 - Consultas resumen y subconsultas bajo especificacion precisa

Se redactaron dos specs precisas. A partir de cada una se pidio a la IA el SQL y se escribio, de forma independiente, una segunda version con distinta estructura. La equivalencia se verifico con EXCEPT en ambas direcciones (0 filas en cada sentido = equivalentes).

4.1 Spec A (resumen) + verificacion

Spec A: "Genera una consulta SQL para Food Store que devuelva, para cada categoria vigente (activo = TRUE), el nombre de la categoria y la cantidad de productos vigentes (activo = TRUE) que tiene, incluyendo las categorias sin productos vigentes con cantidad 0. Orden: cantidad desc, luego nombre. Sin SELECT *. Tablas: categoria, producto."

V1_A (generada por la IA):

```
SELECT c.nombre, COUNT(p.id) AS cantidad_productos
FROM   categoria c
LEFT JOIN producto p ON p.categoria_id = c.id AND p.activo = TRUE
WHERE  c.activo = TRUE
GROUP BY c.id, c.nombre
ORDER BY cantidad_productos DESC, c.nombre;
```

V2_A (alternativa propia - subconsulta correlacionada):

```
SELECT c.nombre,
       (SELECT COUNT(*) FROM producto p
        WHERE p.categoria_id = c.id AND p.activo = TRUE) AS cantidad_productos
FROM   categoria c
WHERE  c.activo = TRUE
ORDER BY cantidad_productos DESC, c.nombre;
```

Verificacion (ejecutada):

```
V1 EXCEPT V2 -> 0 filas   |   V2 EXCEPT V1 -> 0 filas   =>   EQUIVALENTES
```

Resultado real: Bebidas 25.001 productos vigentes · Pizzas 25.001 productos vigentes.

4.2 Spec B (subconsulta) + verificacion

Spec B: "Genera una consulta SQL para Food Store que devuelva los clientes vigentes (activo = TRUE) cuyo gasto total (suma de cantidad x precio_unitario en linea_pedido) supera el gasto promedio de los clientes vigentes, incluyendo los que no tienen pedidos con gasto 0. Columnas: id, nombre, apellido, gasto_total. Orden: gasto_total desc, id asc. Tablas: cliente, pedido, linea_pedido."

V1_B (generada por la IA - agregacion con subconsulta en HAVING):

```
SELECT c.id, c.nombre, c.apellido,
       COALESCE(SUM(lp.cantidad * lp.precio_unitario), 0) AS gasto_total
FROM   cliente c
LEFT JOIN pedido p      ON p.cliente_id = c.id
LEFT JOIN linea_pedido lp ON lp.pedido_id = p.id
WHERE  c.activo = TRUE
GROUP BY c.id, c.nombre, c.apellido
HAVING COALESCE(SUM(lp.cantidad * lp.precio_unitario), 0) > (
    SELECT AVG(g) FROM (
        SELECT COALESCE(SUM(lp2.cantidad * lp2.precio_unitario), 0) AS g
        FROM   cliente c2
        LEFT JOIN pedido p2      ON p2.cliente_id = c2.id
        LEFT JOIN linea_pedido lp2 ON lp2.pedido_id = p2.id
        WHERE  c2.activo = TRUE
        GROUP BY c2.id) AS t)
ORDER BY gasto_total DESC, c.id;
```

V2_B (alternativa propia - CTE + JOIN):

```
WITH gastos AS (
    SELECT c.id, c.nombre, c.apellido,
           COALESCE(SUM(lp.cantidad * lp.precio_unitario), 0) AS gasto_total
    FROM   cliente c
    LEFT JOIN pedido p      ON p.cliente_id = c.id
    LEFT JOIN linea_pedido lp ON lp.pedido_id = p.id
    WHERE  c.activo = TRUE
    GROUP BY c.id, c.nombre, c.apellido)
SELECT g.*
FROM   gastos g
```

```
WHERE g.gasto_total > (SELECT AVG(gasto_total) FROM gastos)
ORDER BY g.gasto_total DESC, g.id;
```

Verificacion (ejecutada):

```
V1 EXCEPT V2 -> 0 filas | V2 EXCEPT V1 -> 0 filas => EQUIVALENTES
```

Resultado real: 9.816 clientes vigentes superan el gasto promedio.

Las cuatro vistas y las verificaciones quedaron ejecutadas y disponibles en 05_consultas_parte4.sql.

Parte 5 - Competencia de optimizacion entre equipos

Consigna comun entregada por la catedra sobre la base masiva: **listado de productos por categoria con filtro de precio y orden, sin indice** (corresponde a la Q1 de la Parte 2). Estrategia del equipo: indice parcial compuesto (*categoria_id, precio*) *WHERE activo = TRUE*, que cubre el filtro de categoria y de vigencia, entrega el rango de precio sin filtro en la tabla y evita el Sort.

Lo que se probó y descarto: (1) el indice simple sobre *categoria_id* ya existia, pero dejaba Bitmap + Sort; (2) un indice compuesto completo (sin la clausula *WHERE* parcial) tambien se descarto: la clausula parcial reduce el indice y permite que el plan no recheck-activo; (3) la IA propuso usar un indice sobre precio solo, se descarto porque pierde el filtro de categoria (doble paso).

Registro de la competencia:

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo despues (ms)	Mejora (x)
Este equipo	Indice parcial compuesto (<i>categoria_id, precio</i>) <i>WHERE activo</i>	8.528	0.213	~40x

Declaracion de Uso de IA (DUIA)

Uso responsable de IA segun la politica de la cathedra: cada uso, su spec, y que se acepto o descarto y por que.

Herramienta	Para que se uso	Prompt / spec (resumen)	Se acepto / se descarto - por que
OpenCode	Parte 1: generar el script de carga masiva	Genera un script SQL (generate_series, sin PL/pgSQL) que inserte 50.000 productos repartidos en las categorias existentes, 20.000 clientes y 200.000 pedidos con lineas, respetando CHECK/UNIQUE/FK, sobre una copia	Aceptado tras lectura linea por linea; se corrigieron errores detectados: huecos de secuencia que rompian FKs y version lenta (reescrita a aritmetica sobre rangos reales).
OpenCode	Parte 2: proponer indices y reescrituras a partir del plan real	Dado este EXPLAIN ANALYZE (nodo, cost, tiempo), proponi cambios que lo mejoren y justificados en el propio plan	Aceptados: indice compuesto parcial (Q1) y GIN pg_trgm (Q2); reescritura sargable (Q3). Descartada: propuesta inicial para Q1 (no mejoraba el plan real) y un indice sobre precio solo.
OpenCode	Parte 3: explicar el plan en lenguaje natural	Explica este plan de EXPLAIN ANALYZE nodo por nodo, sin contexto extra	Aceptada como insumo y luego contrastada: se marcaron 3 afirmaciones correctas, 2 incorrectas (cost vs ms; Execution vs Planning) y 1 imprecisa (orden del GIN).
OpenCode	Parte 4: generar SQL desde especificacion precisa	<Spec A (resumen)...> y <Spec B (subconsulta)...>: genera el SQL, sin solucion previa	Aceptado; se escribio una segunda version propia y se verifico equivalencia formal con EXCEPT (0 filas en ambos sentidos).
Kiro	No utilizado en esta practica	-	-

Entregables y defensa oral

- Script de carga masiva (01_script_carga_masiva.sql) leído, probado y ejecutado bajo protocolo de seguridad.
- Tabla comparativa de la Parte 2 con planes antes/despues completos (secciones 2.3 y 2.4).
- Tabla de lectura critica de la Parte 3.
- Consultas de la Parte 4 (spec + SQL generado + verificacion de equivalencia).
- Registro de la competencia de la Parte 5.
- DUIA completa (pagina anterior).

Defensa oral: cada cambio se puede explicar: Q1 -> el indice compuesto parcial devuelve las filas ya ordenadas y evita el Recheck; Q2 -> GIN de trigramas permite LIKE '%x%' sin barrer las 50.002 filas; Q3 -> EXTRACT tapa la columna (no sargable), el rango equivalente reutiliza idx_pedido_fecha.

Finalizado: 09/09/2026. Base copia: foodstore_tp3 (respaldo previo en backups/).